

Task 2 - Semantic Segmentation, Neuronal Membrane (ISBI2012) with Attention U-Net — CORRECTED

Fixes vs. previous version, per instructor note on Task 2 step 4:

1. **Table 2 metrics now computed on the training set** (all 30 labeled slices), not the 6-slice validation split. Val split is kept only for early-stopping model selection during training, not for final reporting.
2. **Qualitative results now use the actual unlabeled `test_volume`** (no ground truth shown, since none exists for test images), instead of labeled val slices.
3. **Added pixel-level Confusion Matrix and ROC curve**, analogous to the Task 1 (DenseNet121) report, so both notebooks have comparable evaluation figures. Pixels are the unit of classification (membrane=1 vs background=0); computed on the training set to match Table 2's data source.

Everything else (model, loss, training loop, early stopping patience=5) is unchanged.

Model swap note (v2): DeepLabV3+ (lite) underperformed badly (Jaccard 0.13, IoU 0.23, AUC 0.63, early-stopped at epoch 20) — expected on a **24-image training set**. Two things hurt it specifically: (1) its stride-8 dilated backbone throws away spatial resolution a network can't relearn from 24 examples, and (2) `BatchNorm` with `batch_size=2` gives noisy running statistics, a known failure mode for small-batch medical imaging.

Swapped in **Attention U-Net** (Oktay et al., 2018): a full-resolution encoder-decoder with skip connections (so fine membrane boundaries survive downsampling, unlike DeepLab's dilated backbone), attention gates on each skip connection (lets the decoder suppress irrelevant background instead of relying on capacity to learn that from scratch — this is the specific mechanism the original paper showed helps most on small datasets), and **GroupNorm instead of BatchNorm** so training is stable at `batch_size=2`. Still not plain UNet (attention-gated skips, no valid-conv cropping internally, GroupNorm) and not TransUNet (no transformer). Everything else — data pipeline, DiceBCE loss, training loop, early stopping (patience=5), hyperparameters, metrics, and all plots/tables — unchanged.

```
In [1]: !pip install torch torchvision torchaudio scikit-image opencv-python scikit-learn
```

Requirement already satisfied: torch in /opt/conda/lib/python3.13/site-packages (2.13.0)
Requirement already satisfied: torchvision in /opt/conda/lib/python3.13/site-packages (0.28.0)
Requirement already satisfied: torchaudio in /opt/conda/lib/python3.13/site-packages (2.11.0)
Requirement already satisfied: scikit-image in /opt/conda/lib/python3.13/site-packages (0.26.0)
Requirement already satisfied: opencv-python in /opt/conda/lib/python3.13/site-packages (5.0.0.93)
Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.13/site-packages (1.9.0)
Requirement already satisfied: filelock in /opt/conda/lib/python3.13/site-packages (from torch) (3.32.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /opt/conda/lib/python3.13/site-packages (from torch) (4.16.0)
Requirement already satisfied: setuptools>=77.0.3 in /opt/conda/lib/python3.13/site-packages (from torch) (82.0.1)
Requirement already satisfied: sympy>=1.13.3 in /opt/conda/lib/python3.13/site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /opt/conda/lib/python3.13/site-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /opt/conda/lib/python3.13/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /opt/conda/lib/python3.13/site-packages (from torch) (2026.7.0)
Requirement already satisfied: cuda-toolkit==13.0.3 in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.3.0)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /opt/conda/lib/python3.13/site-packages (from torch) (13.3.1)
Requirement already satisfied: nvidia-cudnn-cu13==9.20.0.48 in /opt/conda/lib/python3.13/site-packages (from torch) (9.20.0.48)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.1 in /opt/conda/lib/python3.13/site-packages (from torch) (0.8.1)
Requirement already satisfied: nvidia-nccl-cu13==2.29.7 in /opt/conda/lib/python3.13/site-packages (from torch) (2.29.7)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in /opt/conda/lib/python3.13/site-packages (from torch) (3.4.5)
Requirement already satisfied: triton==3.7.1 in /opt/conda/lib/python3.13/site-packages (from torch) (3.7.1)

Requirement already satisfied: nvidia-cublas==13.1.1.3.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.1.1.3)

Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.88)

Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.96)

Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.0.0.61)

Requirement already satisfied: nvidia-nvjitlink<14,>=13.0.88 in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.3.33)

Requirement already satisfied: nvidia-cufile==1.15.1.6.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (1.15.1.6)

Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.85)

Requirement already satisfied: nvidia-curand==10.4.0.35.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (10.4.0.35)

Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.0.4.66)

Requirement already satisfied: nvidia-cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.6.3.3)

Requirement already satisfied: nvidia-nvtx==13.0.85.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.85)

Requirement already satisfied: cuda-pathfinder>=1.4.2 in /opt/conda/lib/python3.13/site-packages (from cuda-bindings<14,>=13.0.3->torch) (1.6.0)

Requirement already satisfied: numpy in /opt/conda/lib/python3.13/site-packages (from torchvision) (2.5.1)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /opt/conda/lib/python3.13/site-packages (from torchvision) (12.3.0)

Requirement already satisfied: scipy>=1.11.4 in /opt/conda/lib/python3.13/site-packages (from scikit-image) (1.18.0)

Requirement already satisfied: imageio!=2.35.0,>=2.33 in /opt/conda/lib/python3.13/site-packages (from scikit-image) (2.37.4)

Requirement already satisfied: tifffile>=2022.8.12 in /opt/conda/lib/python3.13/site-packages (from scikit-image) (2026.7.31)

Requirement already satisfied: packaging>=21 in /opt/conda/lib/python3.13/site-packages (from scikit-image) (26.2)

Requirement already satisfied: lazy-loader>=0.4 in /opt/conda/lib/python3.13/site-packages (from scikit-image) (0.5)

Requirement already satisfied: joblib>=1.4.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (1.5.3)
 Requirement already satisfied: narwhals>=2.0.1 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (2.24.0)
 Requirement already satisfied: threadpoolctl>=3.5.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (3.6.0)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in /opt/conda/lib/python3.13/site-packages (from sympy>=1.13.3->torch) (1.3.0)
 Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.13/site-packages (from jinja2->torch) (3.0.3)

```
In [2]: import os, glob, re, copy, random
import numpy as np
import cv2 as cv
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
from skimage import io as skio
from sklearn.metrics import confusion_matrix, roc_curve, auc

# --- Model: Attention U-Net (replaces DeepLabV3+ Lite) ---
# Full-resolution encoder-decoder with skip connections + attention gates on each
# (Oktay et al. 2018) – preserves fine spatial detail (unlike DeepLab's dilated
# backbone) and uses GroupNorm instead of BatchNorm for stability at batch_size=
# Not plain UNet (attention-gated skips, GroupNorm, no valid-conv cropping inter
# not TransUNet (no transformer bottleneck).
def center_crop_tensor(t, size):
    h, w = t.shape[-2:]
    top, left = (h - size) // 2, (w - size) // 2
    return t[..., top:top + size, left:left + size]

class DoubleConv(nn.Module):
    def __init__(self, cin, cout, groups=8):
        super().__init__()
        g = min(groups, cout)
        self.block = nn.Sequential(
            nn.Conv2d(cin, cout, 3, padding=1, bias=False),
            nn.GroupNorm(g, cout),
            nn.ReLU(inplace=True),
            nn.Conv2d(cout, cout, 3, padding=1, bias=False),
            nn.GroupNorm(g, cout),
            nn.ReLU(inplace=True),
        )
    def forward(self, x):
        return self.block(x)

class AttentionGate(nn.Module):
    def __init__(self, f_g, f_l, f_int):
        super().__init__()
        self.W_g = nn.Sequential(nn.Conv2d(f_g, f_int, 1, bias=True), nn.GroupNorm(1, f_int))
        self.W_x = nn.Sequential(nn.Conv2d(f_l, f_int, 1, bias=True), nn.GroupNorm(1, f_int))
        self.psi = nn.Sequential(nn.Conv2d(f_int, 1, 1, bias=True), nn.GroupNorm(1, f_int))
        self.relu = nn.ReLU(inplace=True)
    def forward(self, g, x):
        if g.shape[-2:] != x.shape[-2:]:
            g = F.interpolate(g, size=x.shape[-2:], mode='bilinear', align_corners=True)
        psi = self.psi(self.relu(self.W_g(g) + self.W_x(x)))
        return x * psi
```

```

class AttentionUNet(nn.Module):
    def __init__(self, in_ch=1, out_ch=1, out_size=388, base=16):
        super().__init__()
        self.out_size = out_size
        c1, c2, c3, c4, c5 = base, base*2, base*4, base*8, base*16
        self.pool = nn.MaxPool2d(2, ceil_mode=True)

        self.enc1 = DoubleConv(in_ch, c1)
        self.enc2 = DoubleConv(c1, c2)
        self.enc3 = DoubleConv(c2, c3)
        self.enc4 = DoubleConv(c3, c4)
        self.bottleneck = DoubleConv(c4, c5)

        self.up4 = nn.ConvTranspose2d(c5, c4, 2, stride=2)
        self.att4 = AttentionGate(c4, c4, c4 // 2)
        self.dec4 = DoubleConv(c4 + c4, c4)

        self.up3 = nn.ConvTranspose2d(c4, c3, 2, stride=2)
        self.att3 = AttentionGate(c3, c3, c3 // 2)
        self.dec3 = DoubleConv(c3 + c3, c3)

        self.up2 = nn.ConvTranspose2d(c3, c2, 2, stride=2)
        self.att2 = AttentionGate(c2, c2, c2 // 2)
        self.dec2 = DoubleConv(c2 + c2, c2)

        self.up1 = nn.ConvTranspose2d(c2, c1, 2, stride=2)
        self.att1 = AttentionGate(c1, c1, c1 // 2)
        self.dec1 = DoubleConv(c1 + c1, c1)

        self.head = nn.Conv2d(c1, out_ch, kernel_size=1)

    @staticmethod
    def _match(up, skip):
        if up.shape[-2:] != skip.shape[-2:]:
            up = F.interpolate(up, size=skip.shape[-2:], mode='bilinear', align_
        return up

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))
        b = self.bottleneck(self.pool(e4))

        d4 = self._match(self.up4(b), e4)
        d4 = self.dec4(torch.cat([d4, self.att4(d4, e4)], dim=1))

        d3 = self._match(self.up3(d4), e3)
        d3 = self.dec3(torch.cat([d3, self.att3(d3, e3)], dim=1))

        d2 = self._match(self.up2(d3), e2)
        d2 = self.dec2(torch.cat([d2, self.att2(d2, e2)], dim=1))

        d1 = self._match(self.up1(d2), e1)
        d1 = self.dec1(torch.cat([d1, self.att1(d1, e1)], dim=1))

        out = self.head(d1)
        out = self._match(out, x)
        out = center_crop_tensor(out, self.out_size)

```

```

        return torch.sigmoid(out)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
seed = 27
torch.manual_seed(seed); random.seed(seed); np.random.seed(seed)

NET_INPUT_SIZE = 572
NET_OUTPUT_SIZE = 388

# Hyper-parameters
batch_size = 2
n_epochs = 100
learning_rate = 1e-4
patience = 5 # Early Stopping, patience 4/5/6 -> using 5

```

cuda

Data - ISBI2012 Neuronal Membrane dataset

Same loader as before: `./data/train/images`, `./data/train/labels` (30 labeled slices), `./data/test` (unlabeled, no ground truth).

```

In [3]: import glob, re
        from skimage import io as skio

        DATA_DIR = './data'

        def numerical_key(path):
            return int(re.search(r'(\d+)\.tif$', path).group(1))

        train_img_paths = sorted(glob.glob(os.path.join(DATA_DIR, 'train', 'images', '*
        train_mask_paths = sorted(glob.glob(os.path.join(DATA_DIR, 'train', 'labels', '*
        test_img_paths = sorted(glob.glob(os.path.join(DATA_DIR, 'test', '*.tif')), ke

        assert len(train_img_paths) == len(train_mask_paths), "image/label count mismatch
        print(f'train pairs={len(train_img_paths)} test (unlabeled)={len(test_img_paths

        volume = np.stack([skio.imread(p) for p in train_img_paths]) # (30, H, W)
        labels = np.stack([skio.imread(p) for p in train_mask_paths]) # (30, H, W)
        test_volume = np.stack([skio.imread(p) for p in test_img_paths]) # (30, H, W)
        print('volume', volume.shape, 'labels', labels.shape, 'test_volume', test_volume

        # ISBI2012 ground truth convention: membrane pixels are usually 0 (dark), backgr
        # Normalize to {0,1} with foreground(membrane)=1 - verify against a plotted samp
        labels_bin = (labels < 128).astype(np.float32)

        n_total = volume.shape[0]
        idx = list(range(n_total)); random.shuffle(idx)
        n_val = max(4, int(0.2 * n_total))
        val_idx, train_idx = idx[:n_val], idx[n_val:]
        print(f'train slices={len(train_idx)} val slices={len(val_idx)} (val used only

train pairs=30 test (unlabeled)=30
volume (30, 512, 512) labels (30, 512, 512) test_volume (30, 512, 512)
train slices=24 val slices=6 (val used only for early-stopping selection)

In [4]: def pad_to(img, size):
        h, w = img.shape[:2]

```

```

    pad_h, pad_w = max(0, size - h), max(0, size - w)
    top, bottom = pad_h//2, pad_h - pad_h//2
    left, right = pad_w//2, pad_w - pad_w//2
    return cv.copyMakeBorder(img, top, bottom, left, right, cv.BORDER_REFLECT_10)

def center_crop(img, size):
    h, w = img.shape[:2]
    top, left = (h - size)//2, (w - size)//2
    return img[top:top+size, left:left+size]

class MembraneDataset(Dataset):
    def __init__(self, volume, labels_bin, indices, augment=False):
        self.volume, self.labels_bin, self.indices, self.augment = volume, labels_bin, indices, augment
    def __len__(self):
        return len(self.indices)
    def __getitem__(self, i):
        idx = self.indices[i]
        img = self.volume[idx].astype(np.float32) / 255.0
        mask = self.labels_bin[idx]
        img = pad_to(img, NET_INPUT_SIZE)
        mask = pad_to(mask, NET_INPUT_SIZE)
        if self.augment:
            if random.random() > 0.5:
                img, mask = np.fliplr(img).copy(), np.fliplr(mask).copy()
            if random.random() > 0.5:
                img, mask = np.flipud(img).copy(), np.flipud(mask).copy()
        mask = center_crop(mask, NET_OUTPUT_SIZE) # match the network's valid-crop
        img_t = torch.from_numpy(img).unsqueeze(0).float()
        mask_t = torch.from_numpy(mask).unsqueeze(0).float()
        return img_t, mask_t

train_dataset = MembraneDataset(volume, labels_bin, train_idx, augment=True)
val_dataset = MembraneDataset(volume, labels_bin, val_idx, augment=False)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

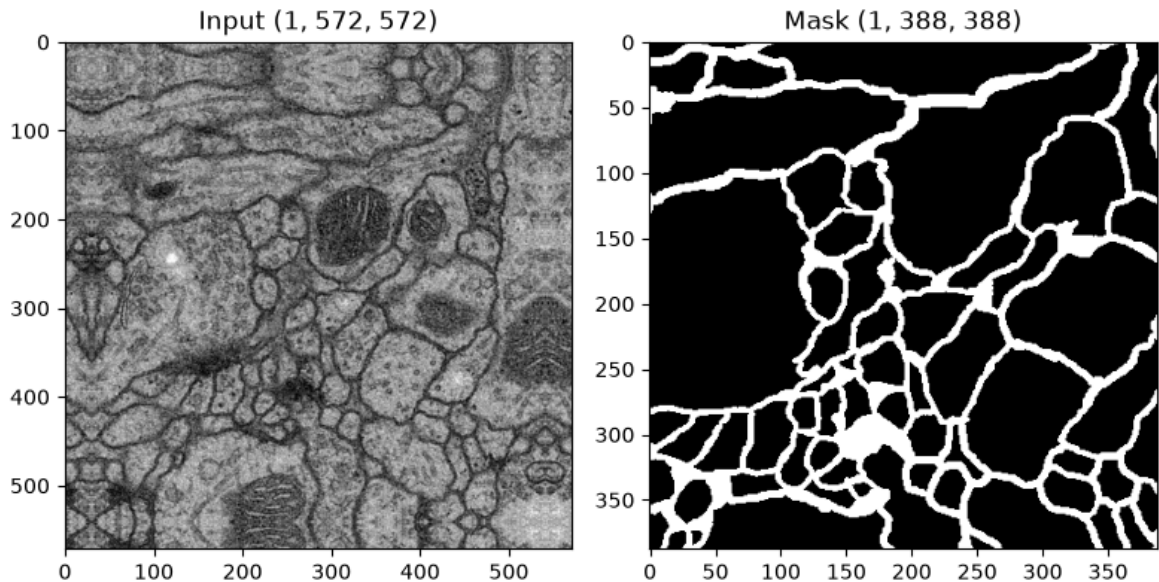
# NEW: full labeled set (all 30 slices, no augmentation) - used for final Table
# and for the pixel-level Confusion Matrix / ROC below, per instructor's note that
# metrics must be reported on the training set (labeled data), not a held-out validation set
full_dataset = MembraneDataset(volume, labels_bin, list(range(n_total)), augment=False)
full_loader = DataLoader(full_dataset, batch_size=batch_size, shuffle=False)

```

```

In [5]: # sanity check: image / mask alignment and the 572->388 crop
img, mask = train_dataset[0]
fig, ax = plt.subplots(1, 2, figsize=(8,4))
ax[0].imshow(img.squeeze(), cmap='gray'); ax[0].set_title(f'Input {tuple(img.shape)}')
ax[1].imshow(mask.squeeze(), cmap='gray'); ax[1].set_title(f'Mask {tuple(mask.shape)}')
plt.tight_layout(); plt.show()

```



Model, loss (DiceBCE), optimizer

```
In [6]: model = AttentionUNet(in_ch=1, out_ch=1, out_size=NET_OUTPUT_SIZE, base=16).to(device)

def dice_loss(pred, target, eps=1e-6):
    pred = pred.contiguous().view(-1)
    target = target.contiguous().view(-1)
    intersection = (pred * target).sum()
    dice = (2*intersection + eps) / (pred.sum() + target.sum() + eps)
    return 1 - dice

def dice_bce_loss(pred, target):
    bce = F.binary_cross_entropy(pred, target)
    dl = dice_loss(pred, target)
    return bce + dl

criterion = dice_bce_loss
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

Segmentation metrics (Precision, Recall/Sensitivity, Accuracy, F1/Dice, Jaccard/IoU)

```
In [7]: def segmentation_metrics(pred, target, thresh=0.5, eps=1e-6):
    pred_bin = (pred > thresh).float()
    tp = (pred_bin * target).sum().item()
    fp = (pred_bin * (1 - target)).sum().item()
    fn = ((1 - pred_bin) * target).sum().item()
    tn = ((1 - pred_bin) * (1 - target)).sum().item()
    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)
    accuracy = (tp + tn) / (tp + tn + fp + fn + eps)
    f1 = 2*tp / (2*tp + fp + fn + eps)
    jaccard = tp / (tp + fp + fn + eps)
    return dict(precision=precision, recall=recall, accuracy=accuracy, f1=f1, jaccard=jaccard)
```

Train / validate with Early Stopping (patience = 5)

Val split is used only to decide when to stop / which checkpoint to keep. It is not used for the final Table 2 numbers.

```
In [8]: def run_epoch(model, dataloader, criterion, optimizer=None):
    is_train = optimizer is not None
    model.train() if is_train else model.eval()
    total_loss, n = 0.0, 0
    metric_accum = {k: 0.0 for k in ['precision', 'recall', 'accuracy', 'f1', 'jaccard']}
    ctx = torch.enable_grad() if is_train else torch.no_grad()
    with ctx:
        for imgs, masks in dataloader:
            imgs, masks = imgs.to(device), masks.to(device)
            preds = model(imgs)
            loss = criterion(preds, masks)
            if is_train:
                optimizer.zero_grad(); loss.backward(); optimizer.step()
            total_loss += loss.item() * imgs.size(0)
            n += imgs.size(0)
            m = segmentation_metrics(preds.detach(), masks.detach())
            for k in metric_accum: metric_accum[k] += m[k] * imgs.size(0)
    avg_loss = total_loss / n
    avg_metrics = {k: v/n for k, v in metric_accum.items()}
    return avg_loss, avg_metrics

Train_Loss, Val_Loss, Val_Jaccard = [], [], []
best_val_loss = float('inf')
best_state = None
epochs_no_improve = 0

for epoch in range(n_epochs):
    train_loss, train_m = run_epoch(model, train_loader, criterion, optimizer)
    val_loss, val_m = run_epoch(model, val_loader, criterion, optimizer=None)
    Train_Loss.append(train_loss); Val_Loss.append(val_loss); Val_Jaccard.append(val_m['jaccard'])
    print(f"Epoch [{epoch+1}/{n_epochs}] Train Loss {train_loss:.4f} | "
          f"Val Loss {val_loss:.4f} Dice(F1) {val_m['f1']:.4f} IoU {val_m['jaccard']:.4f}")

    if val_loss < best_val_loss - 1e-4:
        best_val_loss = val_loss
        best_state = copy.deepcopy(model.state_dict())
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= patience:
            print(f"Early stopping at epoch {epoch+1} (patience={patience})")
            break

model.load_state_dict(best_state)
```

```
Epoch [1/100] Train Loss 1.2704 | Val Loss 1.2067 Dice(F1) 0.5581 IoU 0.3874
Epoch [2/100] Train Loss 1.1606 | Val Loss 1.1291 Dice(F1) 0.6407 IoU 0.4718
Epoch [3/100] Train Loss 1.1032 | Val Loss 1.0841 Dice(F1) 0.6876 IoU 0.5245
Epoch [4/100] Train Loss 1.0578 | Val Loss 1.0427 Dice(F1) 0.7158 IoU 0.5579
Epoch [5/100] Train Loss 1.0140 | Val Loss 1.0029 Dice(F1) 0.7178 IoU 0.5600
Epoch [6/100] Train Loss 0.9756 | Val Loss 0.9659 Dice(F1) 0.7430 IoU 0.5913
Epoch [7/100] Train Loss 0.9446 | Val Loss 0.9503 Dice(F1) 0.7250 IoU 0.5690
Epoch [8/100] Train Loss 0.9250 | Val Loss 0.9164 Dice(F1) 0.7571 IoU 0.6093
Epoch [9/100] Train Loss 0.9006 | Val Loss 0.9019 Dice(F1) 0.7550 IoU 0.6066
Epoch [10/100] Train Loss 0.8830 | Val Loss 0.8803 Dice(F1) 0.7672 IoU 0.6225
Epoch [11/100] Train Loss 0.8688 | Val Loss 0.8738 Dice(F1) 0.7633 IoU 0.6175
```

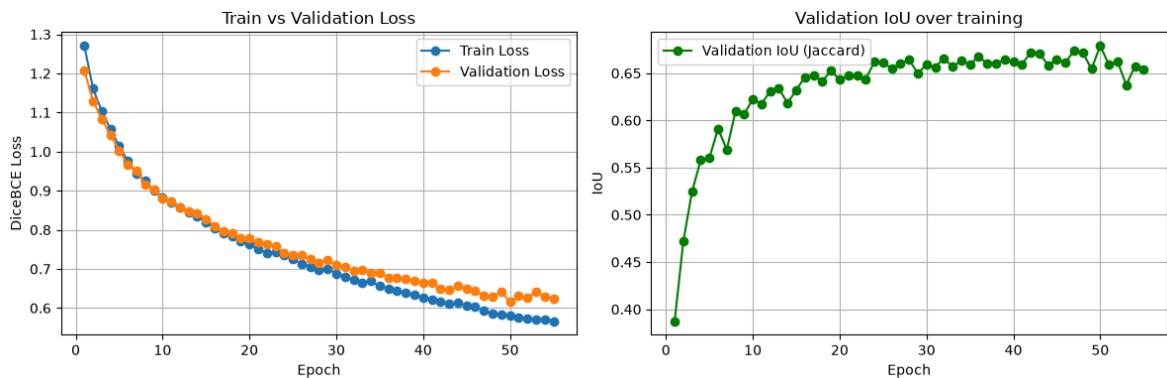
```
Epoch [12/100] Train Loss 0.8585 | Val Loss 0.8578 Dice(F1) 0.7736 IoU 0.6309
Epoch [13/100] Train Loss 0.8449 | Val Loss 0.8465 Dice(F1) 0.7755 IoU 0.6335
Epoch [14/100] Train Loss 0.8336 | Val Loss 0.8421 Dice(F1) 0.7640 IoU 0.6184
Epoch [15/100] Train Loss 0.8197 | Val Loss 0.8274 Dice(F1) 0.7745 IoU 0.6322
Epoch [16/100] Train Loss 0.8050 | Val Loss 0.8094 Dice(F1) 0.7846 IoU 0.6458
Epoch [17/100] Train Loss 0.7910 | Val Loss 0.7952 Dice(F1) 0.7862 IoU 0.6481
Epoch [18/100] Train Loss 0.7831 | Val Loss 0.7911 Dice(F1) 0.7814 IoU 0.6415
Epoch [19/100] Train Loss 0.7713 | Val Loss 0.7784 Dice(F1) 0.7896 IoU 0.6526
Epoch [20/100] Train Loss 0.7624 | Val Loss 0.7783 Dice(F1) 0.7824 IoU 0.6430
Epoch [21/100] Train Loss 0.7512 | Val Loss 0.7687 Dice(F1) 0.7857 IoU 0.6474
Epoch [22/100] Train Loss 0.7409 | Val Loss 0.7622 Dice(F1) 0.7856 IoU 0.6472
Epoch [23/100] Train Loss 0.7440 | Val Loss 0.7583 Dice(F1) 0.7831 IoU 0.6439
Epoch [24/100] Train Loss 0.7360 | Val Loss 0.7404 Dice(F1) 0.7963 IoU 0.6619
Epoch [25/100] Train Loss 0.7246 | Val Loss 0.7352 Dice(F1) 0.7958 IoU 0.6611
Epoch [26/100] Train Loss 0.7130 | Val Loss 0.7346 Dice(F1) 0.7914 IoU 0.6550
Epoch [27/100] Train Loss 0.7052 | Val Loss 0.7243 Dice(F1) 0.7948 IoU 0.6598
Epoch [28/100] Train Loss 0.6968 | Val Loss 0.7159 Dice(F1) 0.7980 IoU 0.6643
Epoch [29/100] Train Loss 0.7010 | Val Loss 0.7229 Dice(F1) 0.7874 IoU 0.6497
Epoch [30/100] Train Loss 0.6883 | Val Loss 0.7094 Dice(F1) 0.7947 IoU 0.6595
Epoch [31/100] Train Loss 0.6802 | Val Loss 0.7055 Dice(F1) 0.7924 IoU 0.6563
Epoch [32/100] Train Loss 0.6710 | Val Loss 0.6948 Dice(F1) 0.7993 IoU 0.6659
Epoch [33/100] Train Loss 0.6650 | Val Loss 0.6981 Dice(F1) 0.7927 IoU 0.6567
Epoch [34/100] Train Loss 0.6683 | Val Loss 0.6890 Dice(F1) 0.7973 IoU 0.6633
Epoch [35/100] Train Loss 0.6561 | Val Loss 0.6909 Dice(F1) 0.7942 IoU 0.6590
Epoch [36/100] Train Loss 0.6499 | Val Loss 0.6770 Dice(F1) 0.8007 IoU 0.6680
Epoch [37/100] Train Loss 0.6435 | Val Loss 0.6775 Dice(F1) 0.7948 IoU 0.6598
Epoch [38/100] Train Loss 0.6384 | Val Loss 0.6741 Dice(F1) 0.7951 IoU 0.6601
Epoch [39/100] Train Loss 0.6349 | Val Loss 0.6684 Dice(F1) 0.7981 IoU 0.6642
Epoch [40/100] Train Loss 0.6277 | Val Loss 0.6655 Dice(F1) 0.7969 IoU 0.6627
Epoch [41/100] Train Loss 0.6225 | Val Loss 0.6649 Dice(F1) 0.7947 IoU 0.6596
Epoch [42/100] Train Loss 0.6170 | Val Loss 0.6496 Dice(F1) 0.8037 IoU 0.6720
Epoch [43/100] Train Loss 0.6112 | Val Loss 0.6478 Dice(F1) 0.8029 IoU 0.6710
Epoch [44/100] Train Loss 0.6135 | Val Loss 0.6557 Dice(F1) 0.7940 IoU 0.6586
Epoch [45/100] Train Loss 0.6064 | Val Loss 0.6481 Dice(F1) 0.7978 IoU 0.6640
Epoch [46/100] Train Loss 0.6030 | Val Loss 0.6438 Dice(F1) 0.7958 IoU 0.6611
Epoch [47/100] Train Loss 0.5925 | Val Loss 0.6313 Dice(F1) 0.8053 IoU 0.6744
Epoch [48/100] Train Loss 0.5871 | Val Loss 0.6301 Dice(F1) 0.8033 IoU 0.6715
Epoch [49/100] Train Loss 0.5825 | Val Loss 0.6416 Dice(F1) 0.7911 IoU 0.6546
Epoch [50/100] Train Loss 0.5797 | Val Loss 0.6163 Dice(F1) 0.8086 IoU 0.6789
Epoch [51/100] Train Loss 0.5766 | Val Loss 0.6310 Dice(F1) 0.7943 IoU 0.6589
Epoch [52/100] Train Loss 0.5731 | Val Loss 0.6256 Dice(F1) 0.7964 IoU 0.6619
Epoch [53/100] Train Loss 0.5715 | Val Loss 0.6424 Dice(F1) 0.7786 IoU 0.6376
Epoch [54/100] Train Loss 0.5717 | Val Loss 0.6295 Dice(F1) 0.7931 IoU 0.6573
Epoch [55/100] Train Loss 0.5665 | Val Loss 0.6235 Dice(F1) 0.7908 IoU 0.6541
Early stopping at epoch 55 (patience=5)
```

Out[8]: <All keys matched successfully>

Graph - Train vs Validation Loss, and IoU over epochs

```
In [9]: epochs_ran = range(1, len(Train_Loss)+1)
fig, axes = plt.subplots(1, 2, figsize=(12,4))
axes[0].plot(epochs_ran, Train_Loss, marker='o', label='Train Loss')
axes[0].plot(epochs_ran, Val_Loss, marker='o', label='Validation Loss')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('DiceBCE Loss'); axes[0].set_title('Train vs Validation Loss')
axes[0].legend(); axes[0].grid(True)
```

```
axes[1].plot(epochs_ran, Val_Jaccard, marker='o', color='green', label='Validati
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('IoU'); axes[1].set_title('Valid
axes[1].legend(); axes[1].grid(True)
plt.tight_layout(); plt.savefig('UNET_training_curves.png', dpi=150); plt.show()
```



FIX 1 - Table 2 metrics reported on the training set

Per instructor note: "Since the Membrane dataset does not provide ground-truth masks for the test images, please report the evaluation metrics only on the training set." All 30 labeled slices are used here (not the 6-slice val split, which was only for early stopping).

```
In [10]: train_set_loss, train_set_metrics = run_epoch(model, full_loader, criterion, opt

import pandas as pd
table2 = pd.DataFrame([
    'Model': 'AttentionUNet (you)', 'Loss': 'DiceBCE',
    'Jaccard': round(train_set_metrics['jaccard'],4),
    'F1 Score': round(train_set_metrics['f1'],4),
    'Recall': round(train_set_metrics['recall'],4),
    'Precision': round(train_set_metrics['precision'],4),
    'Accuracy': round(train_set_metrics['accuracy'],4),
])
table2
```

```
Out[10]:
```

	Model	Loss	Jaccard	F1 Score	Recall	Precision	Accuracy
0	AttentionUNet (you)	DiceBCE	0.7167	0.8348	0.8775	0.7969	0.9209

FIX 3 - Pixel-level Confusion Matrix and ROC curve

Segmentation has no single-label-per-image confusion matrix like Task 1's polyp classifier; the natural analogue is a **pixel-level** confusion matrix (each pixel is a binary sample: membrane vs background), computed on the same training-set data used for Table 2. Storing all ~4.3M pixels is fine for the confusion matrix; ROC is computed on a random subsample of pixels (`MAX_ROC_PIXELS`) purely to keep `sklearn.roc_curve` and the plot fast — this does not change the reported Table 2 numbers above, which come from the full pixel count.

```
In [11]: MAX_ROC_PIXELS = 300_000

def collect_pixel_predictions(model, dataloader):
    model.eval()
    all_probs, all_targets = [], []
    with torch.no_grad():
        for imgs, masks in dataloader:
            imgs = imgs.to(device)
            probs = model(imgs).cpu().numpy().ravel()
            targs = masks.numpy().ravel()
            all_probs.append(probs)
            all_targets.append(targs)
    return np.concatenate(all_probs), np.concatenate(all_targets)

full_probs, full_targets = collect_pixel_predictions(model, full_loader)
full_preds_bin = (full_probs > 0.5).astype(int)
full_targets_int = full_targets.astype(int)

cm = confusion_matrix(full_targets_int, full_preds_bin) # rows=true[Background,
tn, fp, fn, tp = cm.ravel()
print(f'Pixel counts (training set, {len(full_targets_int):,} pixels): TP={tp} F

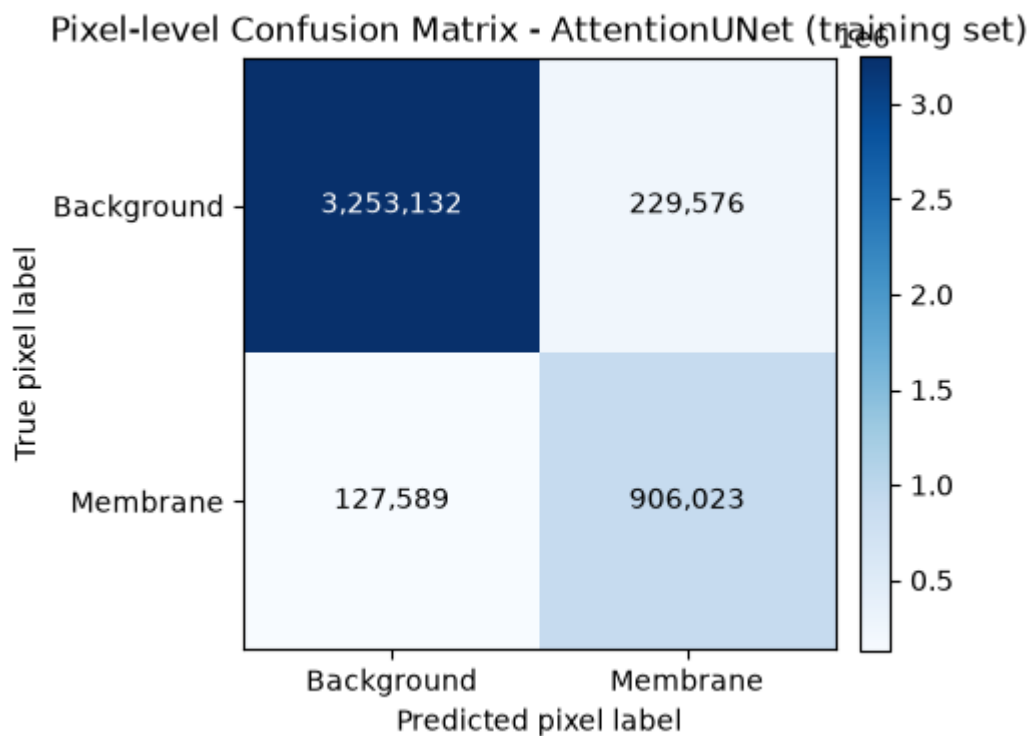
# subsample for ROC curve computation/plot only (cost reasons) - Table 2 numbers
if len(full_probs) > MAX_ROC_PIXELS:
    rng = np.random.default_rng(seed)
    sel = rng.choice(len(full_probs), MAX_ROC_PIXELS, replace=False)
    roc_probs, roc_targets = full_probs[sel], full_targets_int[sel]
else:
    roc_probs, roc_targets = full_probs, full_targets_int

fpr, tpr, _ = roc_curve(roc_targets, roc_probs)
roc_auc = auc(fpr, tpr)
print(f'Pixel-level AUC (training set): {roc_auc:.4f}')
```

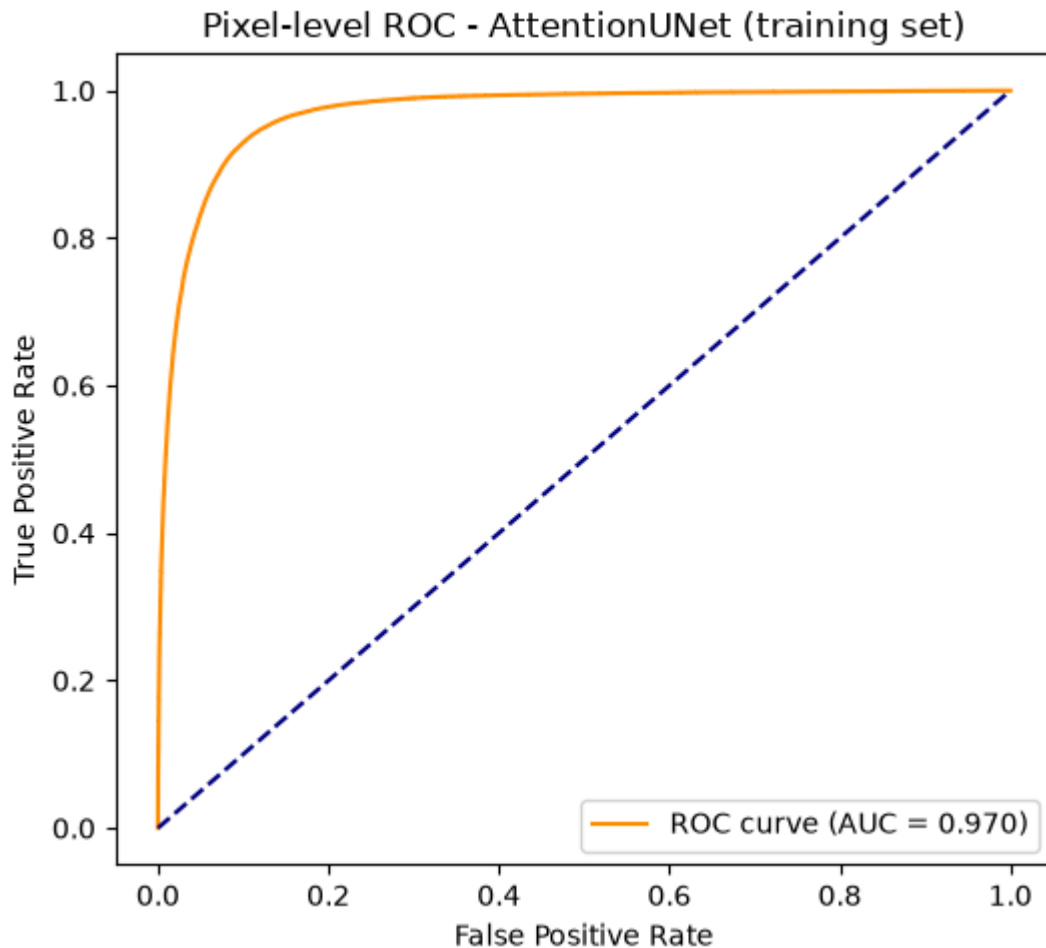
Pixel counts (training set, 4,516,320 pixels): TP=906023 FP=229576 FN=127589 TN=3253132

Pixel-level AUC (training set): 0.9695

```
In [12]: # Graph - Confusion Matrix (pixel-level, training set)
import itertools
fig, ax = plt.subplots(figsize=(5,5))
im = ax.imshow(cm, cmap='Blues')
ax.set_xticks([0,1]); ax.set_xticklabels(['Background','Membrane'])
ax.set_yticks([0,1]); ax.set_yticklabels(['Background','Membrane'])
ax.set_xlabel('Predicted pixel label'); ax.set_ylabel('True pixel label')
ax.set_title('Pixel-level Confusion Matrix - AttentionUNet (training set)')
for i,j in itertools.product(range(2), range(2)):
    ax.text(j, i, f'{cm[i,j]:,}', ha='center', va='center',
            color='white' if cm[i,j] > cm.max()/2 else 'black')
plt.colorbar(im, fraction=0.046, pad=0.04)
plt.tight_layout(); plt.savefig('unet_confusion_matrix.png', dpi=150); plt.show()
```



```
In [13]: # Graph - ROC Curve (pixel-level, training set)
plt.figure(figsize=(5.5,5))
plt.plot(fpr, tpr, color='darkorange', label=f'ROC curve (AUC = {roc_auc:.3f})')
plt.plot([0,1],[0,1], linestyle='--', color='navy')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('Pixel-level ROC - AttentionUNet (training set)')
plt.legend(loc='lower right')
plt.tight_layout(); plt.savefig('unet_roc.png', dpi=150); plt.show()
```

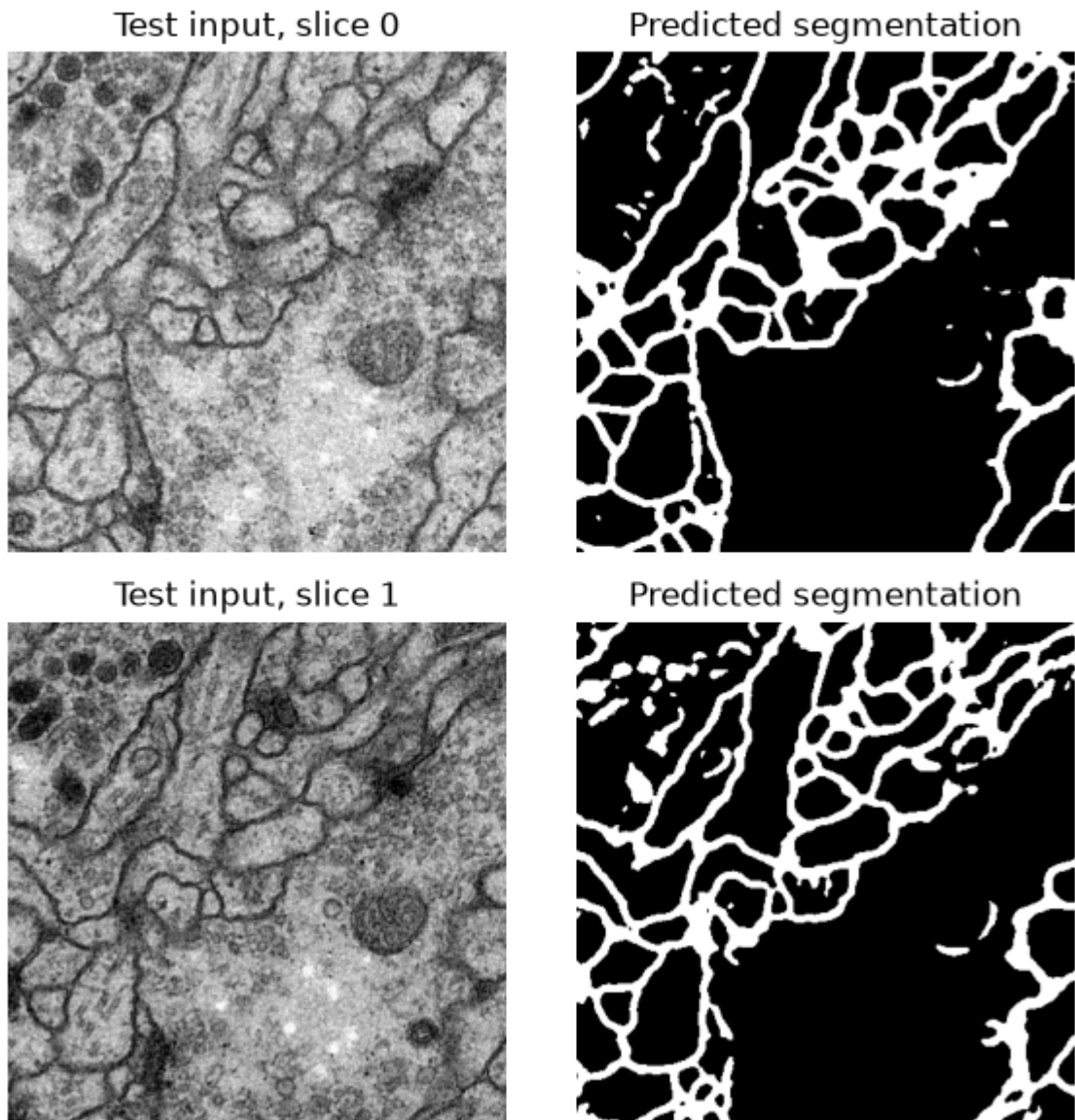


FIX 2 - Required: segmented output of 2 test images

Uses the actual unlabeled `test_volume` (the exercise's real "test images"). No ground-truth panel, since the Membrane test set has none — showing one would have been fabricated. Input + Prediction only, satisfying "for each example, present: the original image, the predicted segmentation."

```
In [14]: model.eval()
n_show = 2
fig, axes = plt.subplots(n_show, 2, figsize=(6, 3*n_show))
with torch.no_grad():
    for row, i in enumerate(range(n_show)):
        img_np = pad_to(test_volume[i].astype(np.float32)/255.0, NET_INPUT_SIZE)
        img_t = torch.from_numpy(img_np).unsqueeze(0).unsqueeze(0).float().to(device)
        pred = model(img_t).squeeze().cpu().numpy()
        pred_bin = (pred > 0.5).astype(np.float32)

        axes[row,0].imshow(center_crop(img_np, NET_OUTPUT_SIZE), cmap='gray')
        axes[row,0].set_title(f'Test input, slice {i}'); axes[row,0].axis('off')
        axes[row,1].imshow(pred_bin, cmap='gray')
        axes[row,1].set_title('Predicted segmentation'); axes[row,1].axis('off')
plt.tight_layout(); plt.savefig('unet_test_predictions.png', dpi=150); plt.show()
```

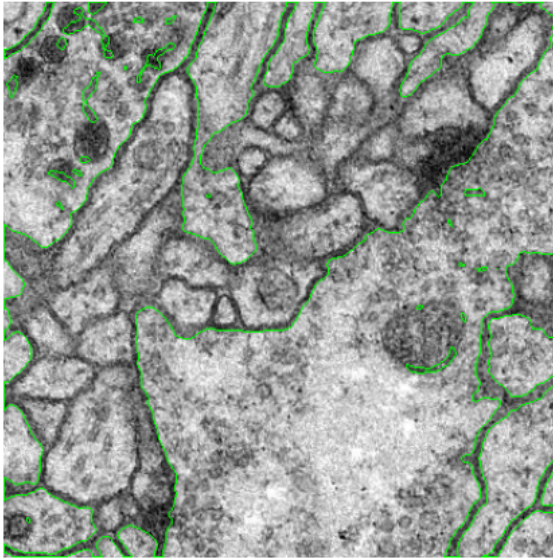


Extra graph for the report - Overlay visualization (now on test images, not val)

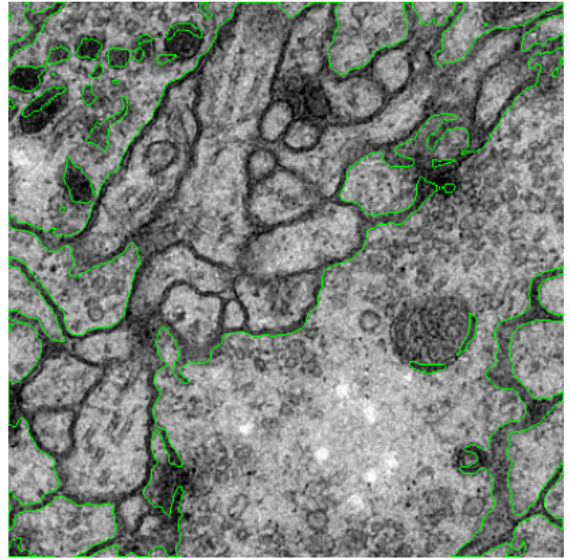
Contour of the predicted membrane on top of the raw EM image, using the same `test_volume` images as above, for consistency with the fix.

```
In [15]: fig, axes = plt.subplots(1, n_show, figsize=(4.5*n_show, 4.5))
if n_show == 1: axes = [axes]
with torch.no_grad():
    for k in range(n_show):
        img_np_full = pad_to(test_volume[k].astype(np.float32)/255.0, NET_INPUT_
img_np = center_crop(img_np_full, NET_OUTPUT_SIZE)
img_t = torch.from_numpy(img_np_full).unsqueeze(0).unsqueeze(0).float()
pred = model(img_t).squeeze().cpu().numpy()
pred_bin = (pred > 0.5).astype(np.uint8)*255
img_rgb = cv.cvtColor((img_np*255).astype(np.uint8), cv.COLOR_GRAY2BGR)
contours, _ = cv.findContours(pred_bin, cv.RETR_EXTERNAL, cv.CHAIN_APPROX
cv.drawContours(img_rgb, contours, -1, (0,255,0), 1)
axes[k].imshow(cv.cvtColor(img_rgb, cv.COLOR_BGR2RGB)); axes[k].axis('of
axes[k].set_title(f'Predicted membrane contour, test slice {k}')
plt.tight_layout(); plt.savefig('unet_overlay.png', dpi=150); plt.show()
```

Predicted membrane contour, test slice 0



Predicted membrane contour, test slice 1



In []: